DSA5106 Deep Learning: Foundations and Techniques

AY2025/26 Sem2 By Zhao Peiduo

Lecture1

A Concrete Definition of Learning A computer program is said to **learn from experience** E with respect to a class of tasks T and performance measure P if its performance on tasks in T, as measured by P, improves with experience E.
Linear Regression as a Canonical Example Task (T):

- Inputs: $x \in \mathbb{R}^d$
- Outputs: $y \in \mathbb{R}$
- Underlying relationship: $y = f^*(x)$
- Goal: predict outputs given inputs

Experience (E):

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad y_i = f^*(x_i) \text{ or } y_i = f^*(x_i) + \varepsilon_i$$

Hypothesis Space Before defining performance, we specify a hypothesis space:

$$\mathcal{H} = \{\text{collection of candidate functions } f\}$$

Performance (P):

$$\text{Distance}(f, f^*)$$

Learning Objective Learning consists of selecting a hypothesis closest to the true function:

$$\hat{f} = \arg\min_{f \in \mathcal{H}} \text{Distance}(f, f^*)$$

Loss Functions and Empirical Risk Distance is defined using a loss function L over the dataset:

$$R_{\text{emp}}(f, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N L(f(x_i), y_i)$$

Example: Squared Loss

$$L(y', y) = \frac{1}{2}(y' - y)^2$$

Empirical Risk Minimization (ERM) Improving performance with experience is formulated as:

$$\hat{f} = \arg\min_{f \in \mathcal{H}} R_{\text{emp}}(f, \mathcal{D})$$

Least Squares Formulation Define the design matrix and output vector:

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

ERM becomes:

$$\min_w \frac{1}{2N} \|Xw - y\|^2$$

Ordinary Least Squares Solution Setting the gradient to zero yields:

$$X^T(Xw - y) = 0$$

$$\hat{w} = (X^T X)^{-1} X^T y, \quad \hat{f}(x) = \hat{w}^T x$$

Extension to Affine Functions Linear models are often insufficient. A simple extension includes a bias term:

$$\mathcal{H} = \{f(x) = w^T x + b \mid w \in \mathbb{R}^d,\ b \in \mathbb{R}\}$$

Extension to Linear Basis Models To model nonlinear relationships, inputs are mapped into features:

$$\mathcal{H} = \left\{ f(x) = w^T \phi(x) = \sum_{i=1}^m w_i \phi_i(x) \right\}$$

The functions $\phi_1, \ldots, \phi_m$ are called **basis functions** or **feature maps**, chosen *a priori*.
Examples of Basis Functions (1D)

- Polynomial basis: $\phi_j(x) = x^j$
- Gaussian basis: $\phi_j(x) = \exp\left(-\frac{(x-m_j)^2}{2s^2}\right)$
- Sigmoid basis: $\phi_j(x) = \sigma\left(\frac{x-m_j}{s}\right)$, $\sigma(b) = \frac{1}{1+e^{-b}}$

Universality of Linear Basis Models
The capacity of linear basis models depends on **the choice of basis functions** and their number. With appropriate basis functions and $m \to \infty$, linear basis models are **universal**: they can approximate arbitrarily well any target function (under mild conditions). For instance:Fourier series and Weierstrass approximation theorem
Least Squares for Linear Basis Models
The least squares solution is obtained by modifying the design matrix. Original design matrix:

$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} \quad \Rightarrow \quad \Phi = \begin{pmatrix} \phi(x_1) \\ \phi(x_2) \\ \vdots \\ \phi(x_N) \end{pmatrix}$$

The empirical risk becomes:

$$R_{\text{emp}}(w) = \frac{1}{2N} \|\Phi w - y\|^2$$

Solution:

$$\hat{w} = (\Phi^T \Phi)^{-1} \Phi^T y$$

Model Capacity and Overfitting Using many basis functions greatly increases model flexibility:

$$\mathcal{H} = \left\{ f(x) = \sum_{j=0}^{99} w_j x^j \;\middle|\; w \in \mathbb{R}^{100} \right\}$$

While such models can fit training data extremely well,they often exhibit **overfitting**: they capture noise rather than the true underlying relationship, leading to poor generalization.
Learning $\neq$ Optimization The goal of learning is not merely minimizing training error, but performing well on **unseen data**.
Empirical Risk Minimization (What we can solve):

$$\min_{f \in \mathcal{H}} R_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N L(f(x_i), f^*(x_i)), \quad x_i \sim \mu$$

Population Risk Minimization (What we want):

$$\min_{f \in \mathcal{H}} R_{\text{pop}}(f) = \mathbb{E}_{x \sim \mu} \left[L(f(x), f^*(x)) \right]$$

The difference between these objectives is known as the **generalization gap**.
Estimating Population Risk
The true data distribution μ is unknown, but we observe samples through the dataset $\mathcal{D}$.
We therefore use a train-test split:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$$

- $\mathcal{D}_{\text{train}}$: used for empirical risk minimization
- $\mathcal{D}_{\text{test}}$: used for final performance evaluation
- The test set must never influence training

Generalization
Generalization is a central challenge in machine learning.Key strategies to improve generalization include:

- Choice of model architecture
- Regularization
- Data augmentation
- Optimization methods

The Argmax View
Regression:

$$y = w^T \phi(x)$$

Classification (K classes):

$$y_i = w_i^T \phi(x), \quad i = 1, \ldots, K$$

Prediction is obtained by:

$$\hat{y} = \arg\max_{i=1,\ldots,K} y_i$$

This formulation is related to, but not equivalent to, support vector machines.
One-Hot Encoding and Softmax
Class labels are represented as one-hot vectors:

$$y_i = (0, \ldots, 1, \ldots, 0) \in \mathbb{R}^K$$

The model outputs class probabilities:

$$f(x_i) = (a_1, \ldots, a_K) \in \mathbb{R}^K$$

Softmax Function

$$s(z)_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

Hypothesis space:

$$\mathcal{H} = \left\{ f(x) = s(W\phi(x)) \;\middle|\; W \in \mathbb{R}^{K \times m} \right\}$$

The output can be interpreted as a probability distribution over K classes.

Loss Functions for Classification

Although classification is naturally defined using zero–one loss, it is not suitable for optimization.
Instead, we use surrogate losses:

$$L : \mathbb{R}^K \times \mathbb{R}^K \to \mathbb{R}_+$$

Desired properties:

- $L(y, y') = 0$ if $y = y'$
- $L(y, y')$ is small if $y \approx y'$
- L is differentiable (almost everywhere)

Examples of Loss Functions for Classification

Even in classification, we can apply the squared loss:

$$L(y, y') = \frac{1}{2} \|y - y'\|^2$$

While this loss is simple and differentiable, it is often poorly aligned with the probabilistic interpretation of classification outputs.
In particular:

- It does not strongly penalize confident but incorrect predictions
- It treats probability vectors as Euclidean points rather than distributions
- It may lead to slower or unstable optimization

Cross-Entropy Loss

A more appropriate loss for probabilistic classification is the cross-entropy loss:

$$L(y, y') = -\sum_{k=1}^{K} y'_k \log y_k$$

When y' is a one-hot encoded label, this loss penalizes low predicted probability assigned to the correct class.

General Perspective

Squared loss and cross-entropy are not the only possible choices.
Any differentiable measure of discrepancy between two probability distributions can be used as a surrogate loss, provided it supports efficient optimization.

Lecture 2

From Linear Basis Models to Neural Networks

Recall that linear basis models can be chosen to be **universal** with appropriate basis functions. Examples of 1D Bases include:

- Polynomial basis: $\phi(x) = (1, x, x^2, x^3, \dots)$
- Fourier basis: $\phi(x) = (1, \cos x, \sin x, \cos 2x, \sin 2x, \dots)$
- Wavelet basis (e.g. Haar wavelets)

Despite universality, empirical performance depends on the target function f^*, the basis choice ϕ, and the available computational budget m. **No fixed basis works well for all targets**.

Adaptive Basis Models

Standard linear basis models take the form:

$$f(x) = w^T \phi(x)$$

In adaptive models, the basis itself is learned:

$$f(x) = w^T \phi(x; \theta)$$

The parameters θ are learned from data to construct a basis well suited to the task.
The functions $\phi_i(\cdot; \theta)$ are also called **feature maps**, as they extract informative representations from data.

Neural Networks as Learned Feature Maps

A neural network can be viewed as a composition. The vector $h \in \mathbb{R}^m$ is the hidden representation, where m is the width of the hidden layer.

$$f = f^{(2)} \circ f^{(1)}$$

$$h = f^{(1)}(x; W, c), \qquad y = f^{(2)}(h; w, b)$$

Need for Nonlinearity

If both layers are linear, the model collapses to a single linear function. Nonlinearity is introduced via an **activation function**:

$$h = g(Wx + c), \qquad y = w^T h + b$$

The activation $g : \mathbb{R} \to \mathbb{R}$ is applied element-wise.

Common Activation Functions

- ReLU: $g(z) = \max(0, z)$
- Sigmoid: $g(z) = \frac{1}{1+e^{-z}}$
- Tanh: $g(z) = \tanh(z)$
- Leaky ReLU: $g(z) = \begin{cases} z, & z \geq 0 \\ \delta z, & z < 0 \end{cases}$

Shallow Neural Networks

A general one-hidden-layer network for regression is:

$$f(x; W, c, w, b) = w^T g(Wx + c) + b$$

where W, w are weights, c, b are biases, and g is the activation function.

Universal Approximation Theorem

A foundational result states that any continuous function on a compact domain can be approximated arbitrarily well by a neural network with a single hidden layer, provided the number of hidden units m is large enough.

Why Neural Networks over Linear Basis Models

A key limitation of fixed linear basis models is the **curse of dimensionality**:the number of required basis functions grows rapidly with input dimension, making learning inefficient. Neural networks mitigate this by learning compact, adaptive representations from data.

Neural Networks for Classification

Neural networks can be adapted for classification in the same way as linear models, by modifying the output layer.
Starting from a regression model:

$$f(x) = w^T g(Wx + c) + b$$

For multi-class classification, we use:

$$f(x) = \sigma\big(V\, g(Wx + c) + b\big), \quad V \in \mathbb{R}^{K \times m}$$

where σ is a normalization function producing class probabilities.

Choice of Output Units

- **Multiclass classification:** softmax output

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- **Multilabel classification:** sigmoid output applied independently

$$\mathrm{sigmoid}(z)_i = \frac{1}{1 + e^{-z_i}}$$

Loss Functions

Learning is performed via **empirical risk minimization**:

$$R_{\mathrm{emp}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} L\big(f^*(x_i), f(x_i; \theta)\big)$$

Regression

For regression problems, we typically use the squared loss:

$$L(y, y') = \frac{1}{2}(y - y')^2$$

Classification Losses

Multiclass classification: Cross-entropy loss

$$L(y, y') = -\sum_j y'_j \log y_j$$

Multilabel classification: Binary cross-entropy loss

$$L(y, y') = -\sum_j \big(y'_j \log y_j + (1 - y'_j) \log(1 - y_j)\big)$$

Why Cross-Entropy over Square Loss

Cross-entropy is preferred for classification because:

- It has a clear probabilistic interpretation (maximum likelihood)
- It provides better numerical stability
- It works naturally with softmax outputs

Empirical Risk Minimization

Training a neural network amounts to solving:

$$\min_\theta R(\theta)$$

Unlike linear regression, closed-form solutions are generally unavailable.

Iterative Optimization

We therefore resort to iterative methods:

$$\theta_{k+1} = \theta_k + F(\theta_k), \qquad R(\theta_{k+1}) \leq R(\theta_k)$$

Gradient Descent

A principled choice for the update direction is the negative gradient:

$$\theta_{k+1} = \theta_k - \epsilon \nabla R(\theta_k)$$

where $\epsilon > 0$ is a step size chosen to ensure a decrease in risk

Gradient Descent via Taylor Expansion

Consider a first-order Taylor expansion of the risk:

$$R(\theta + \epsilon\phi) = R(\theta) + \epsilon \phi^T \nabla R(\theta) + O(\epsilon^2)$$

To decrease the risk, we want the linear term $\phi^T \nabla R(\theta)$ to be as negative as possible. This is achieved by choosing:

$$\phi \propto -\nabla R(\theta)$$

Hence, parameters should be updated in the negative gradient direction:

$$\theta \leftarrow \theta - \epsilon \nabla R(\theta)$$

The Gradient Descent Algorithm
The direction $-\nabla R$ is called the **steepest descent direction**. This leads to the iterative algorithm:

$$\theta_{k+1} = \theta_k - \epsilon \nabla R(\theta_k)$$

This procedure is known as **Gradient Descent (GD)**. The scalar $\epsilon > 0$ is called the **step size** or **learning rate**.
Convergence Analysis (Linear Regression)
Consider gradient descent applied to linear regression. Let w_∞ denote the minimizer. The iteration can be rewritten as:

$$w_{k+1} - w_\infty = A(w_k - w_\infty), \qquad A = I - \epsilon X^T X$$

Defining the error $e_k = w_k - w_\infty$, we obtain:

$$e_k = A^k e_0$$

Let $\{\lambda_i(A)\}$ be the eigenvalues of A. If

$$|\lambda_i(A)| < 1 \quad \forall i,$$

then $e_k \to 0$ and gradient descent converges. This condition is satisfied if:

$$\epsilon < \frac{2}{\max_i \lambda_i(X^T X)}$$

Rate of Convergence
Choosing

$$\epsilon = \frac{1}{\max_i \lambda_i(X^T X)},$$

yields the bound:

$$\|e_k\| \leq \left(1 - \frac{1}{\kappa(X^T X)}\right)^k \|e_0\|$$

where

$$\kappa(X^T X) = \frac{\max_i \lambda_i(X^T X)}{\min_i \lambda_i(X^T X)} \geq 1$$

is the **condition number**. Larger condition numbers imply slower convergence.
Gradient Descent for Neural Networks
For neural networks, empirical risk minimization becomes:

$$\min_{W,c,w,b} \sum_{i=1}^{N} L(f^*(x_i), f(x_i; W, c, w, b))$$

Closed-form solutions are unavailable, so the problem is solved iteratively using gradient descent:

$$W_{k+1} = W_k - \nabla_W R, \quad c_{k+1} = c_k - \nabla_c R, \quad \ldots$$

This forms the basis of training neural networks via first-order optimization.

Lecture 3

From Shallow to Deep Networks
Shallow (One-Hidden-Layer) Neural Network
A shallow neural network with one hidden layer is defined by

$$h = g(Wx + b), \qquad \hat{y} = w^\top h + c.$$

Deep Neural Networks
Deep networks are obtained by repeatedly composing hidden-layer transformations:

$$\begin{aligned} h^{(1)} &= g^{(1)}(W^{(1)} x + b^{(1)}), \\ h^{(2)} &= g^{(2)}(W^{(2)} h^{(1)} + b^{(2)}), \ldots \\ h^{(\ell)} &= g^{(\ell)}(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}), \\ \hat{y} &= w^\top h^{(\ell)} + c. \end{aligned}$$

Why Deep Neural Networks?
Although not fully understood theoretically, deep networks perform well due to:
- Efficient approximation of complex functions (e.g. highly oscillatory ones),
- Improved generalization with proper initialization, training, and regularization,
- Sequential feature extraction providing a strong inductive bias.

Issues with Deep Neural Networks
- Optimization is challenging due to non-convexity,
- Susceptibility to overfitting,
- High computational cost.

Structure of the Empirical Risk
The empirical risk is defined as

$$R(\theta) = \frac{1}{N} \sum_{i=1}^{N} R_i(\theta) = \frac{1}{N} \sum_{i=1}^{N} L\left(y^{(i)}, f(x^{(i)}; \theta)\right).$$

Example: Mean Squared Error (Regression)

$$R(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - w^\top \phi(x^{(i)}; \theta)\right)^2.$$

Computational Complexity of Gradient Descent
Gradient descent minimizes

$$R(\theta) = \frac{1}{N} \sum_{i=1}^{N} R_i(\theta)$$

via the update

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{N} \sum_{i=1}^{N} \nabla R_i(\theta_k).$$

- Per-iteration cost: $\mathcal{O}(N)$,
- Typical dataset sizes: $N \sim 10^4$–10^6,
- Goal: $\mathcal{O}(1)$ cost per iteration.

Stochastic Gradient Algorithm (Robbins & Monro, 1951)
Replace the full gradient with an unbiased estimate.
Gradient Descent (GD)

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{N} \sum_{i=1}^{N} \nabla R_i(\theta_k).$$

Stochastic Gradient Descent (SGD)
Sample

$$\gamma_k \sim \text{Uniform}\{1, \ldots, N\},$$

and update

$$\theta_{k+1} = \theta_k - \epsilon \nabla R_{\gamma_k}(\theta_k).$$

Unbiasedness

$$\mathbb{E}[\theta_{k+1} \mid \theta_k] = \theta_k - \epsilon \frac{1}{N} \sum_{i=1}^{N} \nabla R_i(\theta_k).$$

Mini-Batch SGD
Use a subset $B_k \subset \{1, \ldots, N\}$ with $|B_k| = M$:

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{M} \sum_{j \in B_k} \nabla R_j(\theta_k).$$

Computational Complexity

$$\begin{aligned} \text{GD}: &\ \mathcal{O}(N), \\ \text{SGD}: &\ \mathcal{O}(1), \\ \text{Mini-batch SGD}: &\ \mathcal{O}(M). \end{aligned}$$

Disadvantages of SGD
- GD converges to a stationary point for sufficiently small ϵ,
- If $R(\theta)$ is convex, GD converges to the global minimum,
- Question: what guarantees (if any) hold for SGD?

Example: SGD Dynamics
Recall the gradient descent (GD) dynamics for the quadratic risk

$$R(\theta) = \frac{1}{2} \|\theta\|^2,$$

which yields

$$\theta_k = (1 - \epsilon)^k \theta_0 \to 0 \quad \text{if } 0 < \epsilon < 2.$$

1D Stochastic Variant
Consider a one-dimensional empirical risk:

$$R(\theta) = \frac{1}{N} \sum_{i=1}^{N} R_i(\theta), \qquad R_i(\theta) = \frac{1}{2}(\theta - \theta^{(i)})^2,$$

with assumptions

$$\frac{1}{N}\sum_i \theta^{(i)} = 0, \qquad \frac{1}{N}\sum_i (\theta^{(i)})^2 = 1.$$

The gradient for sample i is

$$\nabla R_i(\theta) = \theta - \theta^{(i)}.$$

SGD Iteration
With $\gamma_k \sim \text{Uniform}\{1, \dots, N\}$,

$$\theta_{k+1} = \theta_k - \epsilon \nabla R_{\gamma_k}(\theta_k) = (1-\epsilon)\theta_k + \epsilon\theta^{(\gamma_k)}.$$

Solving recursively, where the first term is deterministic and the second is random:

$$\theta_k = (1-\epsilon)^k \theta_0 + \epsilon \sum_{j=1}^{k} (1-\epsilon)^{j-1} \theta^{(\gamma_{k-j})},$$

Mean Behavior
Taking expectations,

$$\mathbb{E}[\theta_k] = (1-\epsilon)^k \theta_0,$$

so on average SGD follows the same dynamics as GD.
Variance Behavior
Using the i.i.d. assumption,

$$\mathbb{E}[\theta^{(\gamma_{k-j})}] = 0, \qquad \mathbb{E}[\theta^{(\gamma_{k-j})}\theta^{(\gamma_{k-\ell})}] = \delta_{j\ell},$$

we obtain

$$\mathbb{E}[\theta_k^2] = (1-\epsilon)^{2k}\theta_0^2 + \frac{\epsilon}{2-\epsilon}[1-(1-\epsilon)^{2k}].$$

As $k \to \infty$,

$$\mathbb{E}[\theta_k] \to 0, \qquad \mathbb{E}[\theta_k^2] \to \frac{\epsilon}{2-\epsilon},$$

showing persistent fluctuations for fixed ϵ.
SGD with Decaying Learning Rate
With a varying step size,

$$\theta_{k+1} = \theta_k - \epsilon_k \nabla R_{\gamma_k}(\theta_k),$$

SGD converges if the learning rate decays appropriately. A sufficient condition is

$$\sum_{k=0}^{\infty} \epsilon_k = \infty, \qquad \sum_{k=0}^{\infty} \epsilon_k^2 < \infty.$$

Variants of SGD Common extensions include: **Momentum**, adaptive learning rates (Adagrad, Adadelta), combined methods (Adam, RMSProp), variance reduction (SVRG)
Momentum (Polyak, 1964) Momentum accelerates GD/SGD by remembering past descent directions. For

$$R(\theta) = \frac{1}{2}\theta_1^2 + \frac{\lambda}{2}\theta_2^2, \qquad \lambda \gg 1,$$

the gradient is

$$\nabla R(\theta) = \begin{pmatrix} \theta_1 \\ \lambda\theta_2 \end{pmatrix}.$$

Momentum updates are

$$v_{k+1} = \alpha v_k - \epsilon \nabla R(\theta_k), \qquad \theta_{k+1} = \theta_k + v_{k+1},$$

where v_k is the momentum and $\alpha \in (0,1)$ is the momentum parameter.

- $\alpha = 0$: recovers GD
- $\alpha > 0$: incorporates past directions to reduce zig-zag behavior

Physical Interpretation
Momentum is inspired by Newton's second law:

$$p = mv, \qquad F = m\frac{dv}{dt}.$$

With friction proportional to velocity, which corresponds discretely to momentum SGD.

$$\frac{dv}{dt} = -\frac{\gamma}{m}v + \frac{1}{m}F_{\text{ext}}(\theta), \qquad \frac{d\theta}{dt} = v,$$

SGD in Practice
In practice, SGD almost always uses mini-batches, where $B_k \subset \{1, \dots, N\}$ with $|B_k| = M$:

$$\theta_{k+1} = \theta_k - \epsilon\frac{1}{M}\sum_{j \in B_k} \nabla R_j(\theta_k),$$

The dataset is typically traversed in chunks of size M; one full pass is called an **epoch**.

Computing Gradients
In Gradient Descent (GD) and Stochastic Gradient Descent (SGD), we need to compute gradients of the empirical risk:

$$\nabla R(\theta) \quad \text{or} \quad \nabla R_i(\theta).$$

A key question is how to compute these gradients efficiently for deep neural networks.
Computational Graph
A *computational graph* is a directed graph where:

- Nodes represent variables,
- Directed edges represent operations.

Computational graphs allow us to organize and reuse intermediate computations.
Chain Rule for Scalar Functions
Let $x \in \mathbb{R}$ and f, g be scalar functions. Define

$$y = g(x), \qquad z = f(y) = f(g(x)).$$

Then the chain rule gives

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx} = f'(g(x))\, g'(x).$$

Chain Rule for Vector-Valued Functions
Let

$$f : \mathbb{R}^m \to \mathbb{R}^n, \qquad g : \mathbb{R}^n \to \mathbb{R}, \qquad y = f(x), \qquad z = g(y).$$

Then

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j}\frac{\partial y_j}{\partial x_i}.$$

In vector notation,

$$\nabla_x z = \left(\frac{\partial y}{\partial x}\right)^T \nabla_y z,$$

where $\frac{\partial y}{\partial x}$ is the Jacobian of f.
Why Do We Need an Algorithm?
Although the chain rule tells us how to compute gradients, applying it naively leads to repeated and inefficient computations.
We want an **efficient algorithm** that avoids unnecessary recomputation and scales to deep networks.
Example: Simplest Backpropagation
Consider a 1D linear neural network with one data point:

$$h^{(1)} = w^{(1)}x, \quad h^{(2)} = w^{(2)}h^{(1)}, \quad h^{(3)} = w^{(3)}h^{(2)}, \quad \hat{y} = wh^{(3)}.$$

All variables are scalars. The empirical risk is

$$R(\theta) = R(w^{(1)}, w^{(2)}, w^{(3)}, w) = L(\hat{y}(x;\theta), y).$$

Goal: compute $\frac{\partial R}{\partial w^{(j)}}$ for $j = 1, 2, 3$ and $\frac{\partial R}{\partial w}$ efficiently.
Step 1: Forward Propagation
Given input x, label y, and current weights:

$$h^{(1)} = w^{(1)}x, \quad h^{(2)} = w^{(2)}h^{(1)}, \quad h^{(3)} = w^{(3)}h^{(2)}, \quad \hat{y} = wh^{(3)}.$$

Store intermediate values $h^{(j)}$ for backpropagation.
Step 2: Backward Propagation (Hidden States)
Last layer:

$$\frac{dR}{d\hat{y}} = \frac{\partial}{\partial \hat{y}}L(y, \hat{y}) \to \hat{p}$$

Next layers:

1. $\frac{dR}{dh^{(3)}} = \frac{dR}{d\hat{y}}\frac{d\hat{y}}{dh^{(3)}} = \hat{p}w \to p^{(3)}$
2. $\frac{dR}{dh^{(2)}} = \frac{dR}{dh^{(3)}}\frac{dh^{(3)}}{dh^{(2)}} = p^{(3)}w^{(3)} \to p^{(2)}$
3. $\frac{dR}{dh^{(1)}} = \frac{dR}{dh^{(2)}}\frac{dh^{(2)}}{dh^{(1)}} = p^{(2)}w^{(2)} \to p^{(1)}$

Step 3: Computing Derivatives wrt Parameters
By Chain Rule:

$$\frac{dR}{dw} = \frac{dR}{d\hat{y}}\frac{d\hat{y}}{dw} = \hat{p}h^{(3)}$$

And for $j = 3, 2, 1$:

$$\frac{dR}{dw^{(j)}} = \frac{dR}{dh^{(j)}}\frac{dh^{(j)}}{dw^{(j)}} = p^{(j)}h^{(j-1)} \text{ (where } h^{(0)} := x)$$

Notable Properties

- Two passes: one forward and one backward.
- Total cost: $\mathcal{O}(\#\text{nodes})$ (naïve rule is quadratic).
- Only forward states need to be stored.

Generalizations

- Non-linear and Multi-dimensional cases.
- Tensor inputs/outputs & general DAGs.
- Multi-sample: $\sum \text{backprop}(R_i)$.

Lecture 4

Permutations
A **permutation** of n objects is a one-to-one transformation on these objects. Equivalently, a permutation is a **bijection** on the index set $\{1, 2, \ldots, n\}$.
Permutation Invariance of Hypothesis Spaces
Observe that the fully connected neural network (FCNN) hypothesis space $\mathcal{H}$ satisfies the following invariance property: If $f \in \mathcal{H}$ and p is any permutation of the indices, define

$$f_p(x_1, \ldots, x_d) \equiv f(x_{p(1)}, \ldots, x_{p(d)}).$$

Then, $f_p \in \mathcal{H}$ as well.
Limitation of FCNNs
Permutation invariance implies that FCNNs **discard spatial and temporal structure** in the data. This makes them poorly suited for signals, images, and time series where locality matters.
Averaged Measurement
To reduce noise, we average measurements using a weighting function:

$$s(t) = \int_{-\infty}^{+\infty} x(a)\, w(t-a)\, da.$$

For example, an exponential kernel:

$$w(t-a) = \begin{cases} e^{-(t-a)}, & t \geq a, \\ 0, & t < a. \end{cases}$$

Convolution of Functions
The weighted average above is a special case of a **convolution**. Given two real-valued functions $x(t)$ and $w(t)$, their convolution is

$$s(t) = (x * w)(t) = \int_{-\infty}^{+\infty} x(a)\, w(t-a)\, da.$$

Properties of Convolution
Convolution is **commutative**:

$$x * w = w * x.$$

Conventionally:
- $x(t)$ is called the **signal** or input,
- $w(t)$ is called the **kernel** or filter,
- $s(t) = x * w$ is called the **feature map**.

Discrete Convolutions
In practice, measurements are taken at discrete times: $t = \ldots, -1, 0, 1, 2, \ldots$
The discrete convolution is defined as

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{+\infty} x(a)\, w(t-a).$$

Finite Convolutions and Boundary Conditions
Real signals are finite, so convolutions must be truncated. Common boundary-handling strategies include:
- Circular convolutions,
- Valid convolutions,
- Zero-padded convolutions with `same` padding.

2D Discrete Convolutions
For image and vision tasks, signals are represented as 2D matrices. Given an input image I and kernel K, the 2D convolution is

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)\, K(i-m, j-n).$$

Convolution remains commutative:

$$I * K = K * I.$$

Cross-Correlation vs Convolution
Convolution:

$$S(i,j) = \sum_m \sum_n I(i-m, j-n)\, K(m,n).$$

Cross-correlation:

$$S(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n).$$

While the numerical values may coincide, cross-correlation is not commutative. Deep learning libraries typically implement cross-correlation but refer to it as convolution.
Tensor Convolution
When the input has multiple channels, each filter has the same number of channels. Each channel is convolved separately and the results are summed:

$$\#\text{ output channels} = \#\text{ filters}.$$

Convolution as a Linear Operation
A zero-padded 1D convolution can be written as

$$w * x = A(w)\, x,$$

where $A(w)$ is a structured (Toeplitz) matrix. Thus, convolutions are linear operations.
Convolutional Neural Networks
CNNs replace fully connected layers with convolutions:

$$h^{(i+1)} = g\left(w^{(i+1)} * h^{(i)} + b^{(i+1)}\right).$$

Motivation 1: Sparse Interactions
Due to spatial locality, each neuron interacts with only a small neighborhood, unlike fully connected layers where every node interacts with every other node.
Implicit Connectivity
Although each convolutional layer is local, stacking multiple layers results in long-range dependencies being captured implicitly.
Computational Efficiency
Let image size be $m \times n$ and kernel size be $k \times l$. The computational cost is:
- CNN: $\mathcal{O}(klmn)$,
- FCNN: $\mathcal{O}(m^2 n^2)$.

Since $k, l \ll m, n$, CNNs are significantly more efficient.
Motivation 2: Parameter Sharing
Parameter sharing refers to using the same trainable parameters at multiple spatial locations in a model. An alternative name is **tied weights**. In 1D convolution, this leads to a structured linear operator

$$w * x = A(w)x,$$

$$A(w) = \begin{pmatrix} w_2 & w_3 & 0 & 0 & 0 \\ w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \\ 0 & 0 & 0 & w_1 & w_2 \end{pmatrix}$$

has only **3 degrees of freedom**, compared to a fully connected matrix.
Memory Efficiency
For an image of size $m \times n$ and kernel size $k \times l$:
- With parameter sharing: $\mathcal{O}(k \times l)$ parameters,
- Without sharing: $\mathcal{O}(m \times n \times k \times l)$ parameters.

Motivation 3: Equivariance and Invariance
Let f, g be functions.
- f is **equivariant** w.r.t. g if
$$f(g(x)) = g(f(x)).$$
- f is **invariant** w.r.t. g if
$$f(g(x)) = f(x).$$

Convolutions and Translations
Let T_τ be a translation operator:

$$T_\tau(x)(t) = x(t-\tau).$$

Then for any kernel w,

$$w * T_\tau(x) = T_\tau(w * x),$$

showing that **convolutions are equivariant to translations**.
Compositions of Equivariant Maps
If f_1, f_2 are equivariant w.r.t. g, then so is $f_2 \circ f_1$:

$$f_2 \circ f_1(g(x)) = g(f_2 \circ f_1(x)).$$

Hence each convolutional layer

$$h^{(i+1)} = \sigma(w * h^{(i)} + b)$$

is translation equivariant.
Infinitely Strong Priors

$$\mathcal{H}_{\text{inv}} = \{\, f \;:\; f(T(x)) = f(x) \,\} \quad f^* \in \mathcal{H}_{\text{inv}}$$

Building an Invariant Mapping
Let $f_1, \ldots, f_\ell$ be equivariant w.r.t. g, and let F be invariant w.r.t. g. Then

$$F \circ f_\ell \circ \cdots \circ f_1$$

is invariant w.r.t. g.
A deep CNN of the following form is therefore translation invariant:

$$h^{(i+1)} = \sigma(W * h^{(i)} + b), \quad h^{(0)} = x,$$

$$\hat{y} = \mathbf{1}^\top h^{(\ell)} + c$$

Importance of Translation Invariance
For image classification, the target function f^* must satisfy

$$f^*(T(I)) = f^*(I),$$

since translated images of the same object share the same label. CNNs encode this inductive bias directly.
Pooling Layers
Pooling builds **approximate invariance** to small translations and deformations. Common types include **max** and **mean** pooling.
Basic Structure of CNNs
A convolutional block typically consists of: Convolution (affine transform), nonlinearity (e.g. ReLU), pooling.
Variants of Convolutions
Common extensions include strided convolutions, up/down-sampling operations, unshared convolutions, channel-wise / separable convolutions.

Lecture 5

What is a time series?
Like images, a time series is another type of data with some local structure.
Instead of spatial structure in images, we consider temporal structures.
Representations of a Time Series
We may represent a time series as either:

- A discrete sequence

$$x^{(1)}, x^{(2)}, x^{(3)}, \dots$$

- A continuous sequence

$$\{x^{(t)} : t \geq 0\}$$

Each slice of the time series, $x^{(t)}$, is a vector, or something that can be represented as a vector.
Task I: Sequence Prediction
Given a historical time series data:

$$x^{(1)}, x^{(2)}, \dots x^{(t)}$$

Task is to predict:

- The next data point $x^{(t+1)}$
- A sequence of next data points

$$x^{(t+1)}, x^{(t+2)}, \dots, x^{(t+\tau)}$$

Example applications:

- Stock price prediction, weather forecasting

Task II: Sequence Classification/Regression
Given a time series data:

$$x^{(1)}, x^{(2)}, \dots x^{(t)}$$

Task is to predict a label y of this sequence, which can be either discrete (classification) or continuous (regression).
Example Applications:

- Credit card transaction fraud detection
- Heart arrhythmia detection

Task III: Sequence-to-sequence Modelling
Given a time series data:

$$x^{(1)}, x^{(2)}, \dots x^{(t)}$$

Task is to predict a corresponding time series:

$$y^{(1)}, y^{(2)}, \dots y^{(t)}$$

Example Applications:

- Machine translation
- Continuous health monitoring using wearable devices

Task IV: Sequence Generation
Given a seed sequence:

$$x^{(1)}, x^{(2)}, \dots, x^{(\tau)}$$

Task is to generate a longer sequence starting from this, i.e.

$$x^{(1)}, x^{(2)}, \dots, x^{(\tau)}, x^{(\tau+1)}, x^{(\tau+2)}, \dots$$

according to some distribution.
Example applications:

- Writing poems
- Composing music

The Supervised Learning Problem
Recall that in supervised learning, we define the inputs, outputs and the target function that maps the former to the latter.
For time series modelling, we can define these similarly:

- Inputs: $x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(\tau)}$
- Outputs: $y^{(1)}, y^{(2)}, y^{(3)}, \dots, y^{(\tau)}$
- Target: $\{F_t^*\}$ with

$$y^{(t)} = F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)})$$

Goal: Learn some $\{\hat{F}_t\}$ to approximate $\{F_t^*\}$.
Examples
Task I: Sequence Prediction

$$F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)}) = F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(t)}) = x^{(t+1)}$$

Task II: Sequence Classification/Regression

$$F_\tau^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)}) = y^{(\tau)} = y$$

Task III: Sequence-to-sequence

$$F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)}) = y^{(t)}, \qquad t = 1, 2, \dots$$

or

$$F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(t)}) = y^{(t)}, \qquad t = 1, 2, \dots$$

Dynamical System
A natural way to share parameters in time is to define a dynamical system depending on some parameter:

$$s^{(t+1)} = f(s^{(t)}; \theta)$$

- The vector $s^{(t)}$ is the state of the dynamical system at time t
- θ is a vector of parameters

This is different from "feed-forward" dynamics:

$$h^{(t+1)} = f(h^{(t)}; \theta^{(t)})$$

i.e. the parameter θ is shared in time.
Computational Graph Representation
We can represent the dynamics

$$s^{(t+1)} = f(s^{(t)}; \theta)$$

using the usual computational graph approach.
Unrolling the Dynamics
Of course, we can also unroll the dynamics:

$$s^{(t)} = f(s^{(t-1)}; \theta) = f(f(s^{(t-2)}; \theta); \theta)$$

$$= \cdots = f(f(\cdots f(s^{(0)}; \theta) \cdots ; \theta); \theta).$$

Dynamical Systems with Inputs
Now, we consider a dynamical system with inputs or external forcing:

$$s^{(t)} = f(s^{(t-1)}, x^{(t)}, \theta).$$

Recurrent Neural Networks
The basic architecture of recurrent neural networks uses a forced, hidden dynamical system as a basic hypothesis space:

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}, \theta)$$

$$\hat{y}^{(t)} = g(h^{(t)}, \phi).$$

This implicitly parameterizes

$$\hat{F}_t(x^{(1)}, x^{(2)}, \dots, x^{(t)}; \theta, \phi) = \hat{y}^{(t)}.$$

Simple RNN
To be explicit, we consider the following simple RNN structure:

$$h^{(t)} = \sigma_r(Wh^{(t-1)} + Ux^{(t)} + b)$$

$$\hat{y}^{(t)} = o^{(t)} = \sigma_o(Vh^{(t)} + c).$$

- The trainable parameters are

$$(\theta, \phi) = (W, U, b, V, c).$$

- The recurrent activation σ_r is usually taken to be **tanh**.
- Initialize $h^{(0)} = 0$ by default.
- The output activation function σ_o depends on application.

Tanh Gradients

- Gradient peaks at 1 when $a = 0$.
- Can saturate to ± 1.

Sigmoid Gradients

- Smaller peak gradient of 0.25.
- Non-zero center, outputs saturate to positive numbers, causing systematic bias accumulation.

ReLU Gradients

- Unbounded gradient.
- Dead neurons are dead forever because same weights are shared across time.

Stable Identity Dynamics of tanh
Simple scalar example, $\sigma = \tanh$:

$$h^{(t)} = \sigma(wh^{(t-1)})$$

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = \sigma'(wh^{(t-1)}) \cdot w.$$

When $a \approx 0$, $\sigma(a) \approx a$ and $\sigma'(a) \approx 1$.
If $h^{(t)}$ is small and $w \approx 1$: $h^{(t)} \approx h^{(t-1)}$, Gradients approx. 1
Loss Functions
For single-prediction tasks (Tasks I and II), the loss function may be defined only at the end of the sequence, e.g.

$$L(y, \hat{y}^{(\tau)}) = \frac{1}{2} \|y - \hat{y}^{(\tau)}\|^2.$$

For sequence-prediction tasks (Task III), we may take the sum:

$$\sum_{t=1}^{\tau} L(y^{(t)}, \hat{y}^{(t)}) = \frac{1}{2} \sum_{t=1}^{\tau} \|y^{(t)} - \hat{y}^{(t)}\|^2.$$

Why a hidden dynamical system?
Consider instead the naive approach of just using FCNN to map each input to each output:

$$\hat{y}^{(t)} = \text{FCNN}(x^{(t)}).$$

What is wrong?
- The prediction at time t only depends on $x^{(t)}$.
- This cannot model systems with memory, no matter how complex FCNN is.

Unrolling Computational Graph
Once we unroll the computational graph, then RNN is just a parameter-tied feed-forward NN, thus we can apply back-propagation algorithm.
General BPTT
General Back-propagation through time (BPTT) basically does the same:
- Unroll computational graph
- Compute "deltas" represented by

$$p^{(t)} := \nabla_{h^{(t)}} L$$

- Compute derivatives with respect to weights by **summing** over contributions at each time step

Only the last step differs from the feed-forward case, due to parameter sharing in time.
Gradient Explosion and Vanishing
A big problem that plagues RNNs is the difficulty to train them.
One is often faced with a dilemma for large τ:
- Gradient explosion: for some parameter values, the gradients computed using the RNN diverges
- Gradient vanishing: for other parameter values, the gradients computed using the RNN gives vanishing weight to faraway inputs

Gating and Long-Term Dependencies
One of the most effective ways to overcome the inability for RNNs to learn long-term dependencies is their gated extensions.
The key ideas:
- Design RNN variants where gradients neither vanish nor explode
- Achieve this by the construction of "trainable gates", which controls the flow of information
  - Gates control the accumulation of information
  - When accumulated information is no-longer required, we just forget them

RNN Cells
RNNs operate by feeding the input and previous hidden states to a function to give the next hidden state, e.g.

$$h^{(t)} = \tanh(Wh^{(t-1)} + Ux^{(t)} + b)$$

We can abstract this operation as a black-box cell A.
Long Short Term Memory (LSTM)
LSTM [Hochreiter and Schmidhuber, 1997] is one of the most commonly used gated RNNs.
The Cell State
The cell state C_t is another hidden variable that is designed to flow through time with minimal interruptions.
Gates
The LSTM interacts with the cell state using gates, there are just layers with sigmoid outputs

$$u \mapsto \sigma(Wu + b),$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Multiplying the cell state with the above makes a modification to it.
Summary: LSTM Model (In our Notation)

$$f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f)$$

$$i^{(t)} = \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i)$$

$$o^{(t)} = \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o)$$

$$C^{(t)} = f_t \cdot C_{t-1} + i_t \cdot \sigma_c(W_c h^{(t-1)} + U_c x^{(t)} + b_c)$$

$$h^{(t)} = o^{(t)} \cdot \sigma_h(C^{(t)})$$

Gates: $f^{(t)}$, $i^{(t)}$, $o^{(t)}$
Cell state: $C^{(t)}$
Hidden/RNN state: $h^{(t)}$
Variants of the LSTM Architecture
There are many variants:
- Peephole connections
- Coupled input/forget gates
- Gated Recurrent Unit (GRU)

Deep Recurrent Neural Networks
One may observe that the RNN, even in its simplest form, is already deep in the time direction.
However, each time-step relied on a single layer of FCNN-like structure.
Deep RNNs generalize this by using a deep NN for the recurrent step.
The Basic Architecture
Shallow/Simple RNN:

$$h^{(t)} = \sigma_r(Wh^{(t-1)} + Ux^{(t)} + b)$$

$$\hat{y}^{(t)} = \sigma_o(Vh^{(t)} + c)$$

Deep RNN (One variant):

$$h^{(t)} = \sigma_r(Wh^{(t-1)} + Ux^{(t)} + b)$$

$$z^{(t)} = \sigma_r(Sz^{(t-1)} + Th^{(t)} + a)$$

$$\hat{y}^{(t)} = \sigma_o(Vz^{(t)} + c)$$

Other Ways to Obtain Deep RNN Models
- (a) Using stacked hidden/cell states
- (b) Using MLPs for connections

Other Important Architectures for Sequence Modelling
There are a number of other architectures for time-series modelling that are useful to know:
- Bi-directional RNNs
- Sequence-to-sequence autoencoders
- Attention mechanisms and the transformer network

Lecture 6

Regularization Regularization is a general technique: Any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error. In other words, we trade **some bias to lower variance**.
Parameter Norm Penalties Empirical risk minimization

$$\hat{\theta} = \arg\min_{\theta} R(\theta; X, y)$$

A parameter norm penalty modifies this to

$$\tilde{R}(\theta; X, y) = R(\theta; X, y) + \alpha\Omega(\theta)$$

where
- Ω is called a regularizer
- $\alpha \geq 0$ is the strength (coefficient) of regularization

L^2 Regularization
The simplest parameter norm regularization is the L^2 regularizer

$$\Omega(\theta) = \frac{1}{2}\|\theta\|^2 = \frac{1}{2}\sum_i \theta_i^2$$

so that

$$\tilde{R}(\theta; X, y) = R(\theta; X, y) + \frac{1}{2}\alpha\|\theta\|^2$$

and

$$\hat{\theta} = \arg\min_{\theta} \tilde{R}(\theta; X, y)$$

Example: Linear Regression
L^2–regularized linear model

$$R(w; X, y) = \frac{1}{2}\|Xw - y\|^2 + \frac{1}{2}\alpha\|w\|^2$$

This is called **ridge regression** (Tikhonov regularization).
Solution:

$$\hat{w} = (X^T X + \alpha I)^{-1} X^T y$$

Compare with unregularized version:

$$H = X^T X, \qquad \tilde{H} = H + \alpha I$$

Effect of the Regularization
Recall

$$\frac{1}{m} H_{ij} = \frac{1}{m}(X^T X)_{ij} = \frac{1}{m}\sum_k x_i^{(k)} x_j^{(k)}$$

which is the covariance matrix of the data.
The principal directions are eigenvectors $\{u_i\}$ of H with eigenvalues $\{\lambda_i\}$.
Let

$$Hu_i = \lambda_i u_i$$

Expand

$$X^T y = \sum_i \beta_i u_i$$

Unregularized:

$$\hat{w} = \sum_i \frac{\beta_i}{\lambda_i} u_i$$

Regularized:

$$\hat{w} = \sum_i \frac{\beta_i}{\lambda_i + \alpha} u_i$$

Effect of L^2 Regularization

- If $\lambda_i \gg \alpha$, then $\lambda_i + \alpha \approx \lambda_i$
- If $\lambda_i \ll \alpha$, then $\lambda_i + \alpha \approx \alpha$
- The regularization removes the influence of the data in directions of small variance.

Additional Remarks on L^2 Regularization

L^2 regularization is also called **weight decay**.
Gradient descent:

$$\theta_{k+1} = \theta_k - \epsilon \nabla \tilde{R}(\theta_k) = \theta_k - \epsilon \nabla R(\theta_k) - \epsilon \alpha \theta_k$$

If $R = C$ (constant):

$$\theta_{k+1} = (1 - \epsilon\alpha)\theta_k$$

L^1 Regularization

Another type of parameter norm regularization is L^1:

$$\Omega(\theta) = \|\theta\|_1 = \sum_i |\theta_i|$$

L^1 vs L^2 Regularization

Consider minimizing

$$R(\theta) = \frac{1}{2} \sum_i \lambda_i (\theta_i - \theta_i^*)^2$$

where $\theta^* \in \mathbb{R}^m$ is fixed.

Solution for L^2 regularized case

$$\nabla \tilde{R}(\hat{\theta}) = 0 \Rightarrow \lambda_i(\hat{\theta}_i - \theta_i^*) + \alpha \hat{\theta}_i = 0 \Rightarrow \hat{\theta}_i = \frac{\lambda_i}{\lambda_i + \alpha} \theta_i^*$$

Solution for L^1 regularized case

$$\tilde{R}(\theta) = \sum_i R_i(\theta_i)$$

with

$$R_i(\theta_i) = \frac{1}{2} \lambda_i (\theta_i - \theta_i^*)^2 + \alpha |\theta_i|$$

Thus we can minimize each component of θ independently:

$$\hat{\theta}_i = \begin{cases} \theta_i^* - \text{Sign}(\theta_i^*) \frac{\alpha}{\lambda_i}, & |\theta_i^*| > \frac{\alpha}{\lambda_i} \\ 0, & |\theta_i^*| \leq \frac{\alpha}{\lambda_i} \end{cases}$$

Unregularized

$$\hat{\theta}_i = \theta_i^*$$

L^2-Regularized

$$\hat{\theta}_i = \frac{\lambda_i}{\lambda_i + \alpha} \theta_i^*$$

L^1-Regularized

$$\hat{\theta}_i = \begin{cases} \theta_i^* - \text{Sign}(\theta_i^*) \frac{\alpha}{\lambda_i}, & |\theta_i^*| > \frac{\alpha}{\lambda_i} \\ 0, & |\theta_i^*| \leq \frac{\alpha}{\lambda_i} \end{cases}$$

Comparison between L^2 and L^1 regularization

Common to both:

- As α increases from 0 to ∞, the solution $\hat{\theta}$ is pushed from θ^* toward 0
- Higher variance components of the data (large λ_i) are less affected than low variance components (small λ_i)

Differences:

- L^1 is "hard": if λ_i is small enough the corresponding weight becomes exactly 0
- L^1 penalty therefore induces sparsity

Remarks on L^1 and L^2 Regularization

- Sparsity can be used for feature selection (principle behind LASSO)
- From a Bayesian MAP perspective, parameter norm regularization corresponds to priors on parameters

$$L^2: \quad p(\theta) \propto \exp\left(-\frac{\alpha}{2} \|\theta\|^2\right)$$

$$L^1: \quad p(\theta) \propto \exp(-\alpha \|\theta\|_1)$$

Elastic Net combines both L^1 and L^2 penalties.

Remarks on Regularizing Neural Networks

So far examples were linear models, but regularization behaves similarly for nonlinear neural networks.
Consider a neural network layer:

$$\sigma(Wx + b)$$

Usually we regularize weights W but not bias b.
Bias terms only shift activations and do not need to be small.
Different layers may use different regularization strengths.

Training vs Validation Loss

Often in practice:

- Training loss continues decreasing
- Validation loss stays constant or increases

This indicates overfitting.

Early Stopping Variants

Early stopping requires a validation set.
Two strategies:

- Retrain on the full dataset after identifying optimal number of epochs
- Continue training on full dataset using the validation loss threshold

Algorithm: Early Stopping (Retrain Variant)

Let $X^{(train)}, y^{(train)}$ be the training set.
Split into

$$(X^{(subtrain)}, X^{(valid)}), \quad (y^{(subtrain)}, y^{(valid)})$$

Run early stopping using $X^{(subtrain)}$ and $y^{(subtrain)}$ for training and validation data for monitoring.
This returns optimal number of steps i^*.
Reinitialize parameters θ.
Train on full training data for i^* steps.

Algorithm: Early Stopping (Continuation Variant)

Let $X^{(train)}, y^{(train)}$ be the training set.
Split into

$$(X^{(subtrain)}, X^{(valid)}), \quad (y^{(subtrain)}, y^{(valid)})$$

Run early stopping to update θ.
Set

$$\epsilon \leftarrow J(\theta, X^{(subtrain)}, y^{(subtrain)})$$

While

$$J(\theta, X^{(valid)}, y^{(valid)}) > \epsilon$$

train on full dataset for n steps.

Example: Early Stopping vs Explicit Regularization

Consider the one-dimensional loss

$$R(\theta) = \frac{1}{2} \lambda (\theta - \theta^*)^2$$

Gradient descent

$$\theta_{k+1} = \theta_k - \epsilon\lambda(\theta_k - \theta^*) = (1 - \epsilon\lambda)\theta_k + \epsilon\lambda\theta^*$$

Thus

$$\theta_k = (1 - \epsilon\lambda)^k \theta_0 + \left(1 - (1 - \epsilon\lambda)^k\right) \theta^*$$

If we stop at iteration τ:

$$\hat{\theta} = (1 - \epsilon\lambda)^\tau \theta_0 + \left(1 - (1 - \epsilon\lambda)^\tau\right) \theta^*$$

Hence $\hat{\theta}$ is a weighted average of the initial parameter θ_0 and the optimum θ^*.
Let us now look at a variant of L^2 regularized GD

$$\tilde{R}(\theta) = R(\theta) + \frac{1}{2} \alpha (\theta - \theta_0)^2$$

That is, we apply the regularization centered at θ_0.
Then the solution is

$$\hat{\theta} = \frac{\alpha}{\alpha + \lambda} \theta_0 + \left(1 - \frac{\alpha}{\alpha + \lambda}\right) \theta^*$$

Compare with early stopping

$$\hat{\theta} = (1 - \epsilon\lambda)^\tau \theta_0 + \left(1 - (1 - \epsilon\lambda)^\tau\right) \theta^*$$

In other words, early stopping at time τ is equivalent to L^2 regularization with strength

$$\alpha = \frac{\lambda(1 - \epsilon\lambda)^\tau}{1 - (1 - \epsilon\lambda)^\tau}$$

Remarks on Early Stopping

Advantages:

- Very easy to implement
- Effectively a regularization with a penalty depending on the distance from initial condition

$$\frac{1}{2} \alpha (\theta - \theta_0)^2$$

- Implicit regularization, no need to choose α.

Different ways to add Noise
The basic rationale behind adding noise to regularize models:

Our model should be robust to noise

Different ways to add noise:
- Inputs
- Outputs / labels
- Weights

—

Adding Noise to Inputs
This is heuristically equivalent to placing the prior:

The output is insensitive to random perturbations of the input

Again, this is a form of implicit regularization.
Increasing noise, no change in label.

—

Example: Linear Regression with Noisy Inputs
Recall: least squares problem minimizes

$$R(w) = \frac{1}{2}\|Xw - y\|^2$$

Suppose we add noise to the inputs, which replaces

$$X \mapsto \tilde{X} = X + Z,$$

$$Z^{(i)} \sim N(0, \delta I)$$

where $Z^{(i)}$ denotes the ith row of Z.
Then

$$\tilde{R}(w) = \frac{1}{2}\|\tilde{X}w - y\|^2 = \frac{1}{2}\|Xw - y\|^2 + \frac{1}{2}\|Zw\|^2 + (Zw)^T(Xw - y)$$

And so

$$\mathbb{E}_Z[\tilde{R}(w)] = R(w) + \frac{1}{2}\delta N\|w\|^2$$

That is, adding noise to the input here is equivalent to L^2 regularization.
This is also heuristically true in general.

Adding Noise to the Output
The rationale behind injecting noise to the output varies from application to application.
Examples:
- For classification problems, the given labels may have a small probability of being wrong
- The labels / outputs could be randomly drawn according to some distribution

In these cases, it may be advantageous to model such noise explicitly.

Label Smoothing
An oft-used example of output noise injection is label smoothing.
For a classification problem with a one-hot label, we can smooth the label by placing

$$1 \to 1 - \alpha$$

and

$$0 \to \frac{\alpha}{K - 1}$$

Example:

$$y = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \quad \to \quad y = \begin{pmatrix} \alpha/3 \\ 1 - \alpha \\ \alpha/3 \\ \alpha/3 \end{pmatrix}$$

Adding Noise to the Weights
Noise can also be added to the weights during training.
Consider one input-output pair (x, y).
Suppose we have a neural network which makes the prediction

$$\hat{y} = f(x, \theta)$$

Now, we add a small perturbation $\delta\phi$ ($\delta \ll 1$) to θ,

$$\phi \sim N(0, I)$$

Then the risk (averaged over ϕ) is

$$\mathbb{E}_\phi \tilde{R}(\theta) = \frac{1}{2}\mathbb{E}_\phi\,(f(x, \theta + \delta\phi) - y)^2$$

$$= \frac{1}{2}(f(x, \theta) - y)^2 + \frac{m\delta^2}{2}\|\nabla_\theta f(x, \theta)\|^2 + \cdots$$

That is, this penalizes variations of the neural network predictions with respect to the weights.